Function Types Applicable to the Scheme in the Paper A New Framework for Efficient Multivariate Functional Bootstrapping

1 Why this note exists

This note records which finite functions have actually compressed well under the ideas used in the paper. It is not a theorem claiming that every member of a named family will compress. We tried more functions than are shown in the main tables, kept the successful parameter sets, and left all rejected rows in the raw CSV and JSON files.

The common opportunity is simple:

$$|\mathrm{Im}\, f| \ll |\mathrm{Dom}\, f|.$$

Many direct-table inputs then share one output. The work is to find a small intermediate value that identifies the right output class. A small range is the reason to look for compression; it is not, by itself, a construction or a speed proof.

The conclusions below are deliberately heuristic and tied to the listed finite parameters. This document may be enlarged later when more functions and ciphertext implementations have been tested.

2 Notation and what the numbers mean

We use the notation of the paper. Thus $[t] = \{0, 1, \ldots, t-1\}$, the usual case is

$$f : [t]^\ell \longrightarrow [t],$$

and the paper searches for unary maps $p_i : [t] \to \mathbb{Z}$ and an outer map Q such that

$$b(x_1, \ldots, x_\ell) = \sum_{i=1}^{\ell} p_i(x_i), \qquad f(x_1, \ldots, x_\ell) = Q(b(x_1, \ldots, x_\ell)).$$

Its outer span is

$$E = \max_{[t]^\ell} b - \min_{[t]^\ell} b,$$

so the paper's direct and decomposed record counts are t^ℓ and $\ell t + E + 1$, respectively.

Some experiments below have unequal input alphabets, for example two binary strings or $m \in [0, t-1]$, $d \in [1, t-1]$. Only in those cases we use the more general notation $D = D_1 \times \cdots \times D_\ell$; the direct count is then $|D|$. There is no separate output symbol: "#out" in the tables means the number of different values of f on the stated domain.

For a rounded additive construction, the program forms integer unary tables p_i, enumerates every input, and accepts an outer table only when

$$b(x) = b(x') \quad \Longrightarrow \quad f(x) = f(x')$$

for all tested x, x'. For comparison networks, histograms, and packed counts, "Ours" is the sum of the consecutive unary-LUT intervals used by the stated evaluator. The same LUT may be reused in an implementation, but repeated evaluations are counted conservatively. Public additions and public-constant multiplications are not LUT evaluations.

These are structural record counts, not ciphertext timings. Parameter growth, noise, packing, key switching, memory access, and parallelism can change the practical result. In both tables, "Direct" is the number of records in one full LUT on the original Cartesian input domain. "Gain" is

$$\text{Direct}/\text{Ours}.$$

"LUT evals." is the number of functional-bootstrap LUT evaluations, not circuit depth: independent evaluations are still counted separately. In Table 1, M is the integer scale used to round the unary features. In Table 2, "Alt." is the record count of the explicit intermediate-value route described below, and "Alt./ours" compares that route with the paper's route.

3 How the experiments were run

The deterministic program is `expanded_function_study/run_experiments.py`. It uses only the Python standard library. For every row it enumerates the complete finite domain, evaluates the direct definition and the proposed evaluator, checks every output, and writes the result to `results/all_results.csv` and `results/results.json`.

The base run contains 76 parameterized candidates. Forty-five satisfy its mechanical rule "maximum error zero and Direct/Ours at least 2." Simple affine sums followed by clipping or thresholding are still omitted because they do not need the nonlinear decomposition studied here.

This revision also runs `expanded_function_study/compare_intermediate_routes.py`. It checks 59 function/parameter rows against an explicit competing route that first forms an algebraic, sorted, or histogram intermediate and then uses one final LUT. The competitor is deliberately favoured: its ciphertext multiplications are reported but assigned zero LUT records. A row is retained only when every finite input is correct, Direct/Ours is at least 2, and Ours is strictly smaller than the competing record count. The script retains 19 elementary rows and 26 further non-trivial rows; all 14 rejected comparisons remain in the CSV and JSON output.

For the transcendental ratios, every direct output was computed independently at 80 and 110 decimal digits. The two integer results had to agree at every input. Exact integer identities, such as $\log 9/\log 3 = 2$, are handled explicitly by snapping only after agreement to 65 decimal places. This is a high-precision finite check, not a symbolic proof about transcendental constants.

4 Low-depth and deeper elementary compositions

4.1 The finite transform pattern

A useful finite-domain pattern uses logarithms to turn products or ratios into additive features and then applies an outer reconstruction table. The same search pattern is useful for a somewhat deeper expression

$$f(x_1, \dots, x_\ell) = F\big(\phi_1(x_1) + \cdots + \phi_\ell(x_\ell)\big).$$

For a scale M, the program rounds the unary features to integer tables, adds them, and checks whether the resulting code separates all output classes. The accepted outer LUT is defined on the whole interval from the smallest to the largest code. Consequently, a function is rejected when the precision needed for separation makes that interval too large, even if its output range is small.

4.2 Retained experiments

Table 1 contains every retained elementary-composition row. The notation and record count are the same as in Section 2. In the bivariate ratio rows $x, y \in \{1, \dots, t-1\}$, so the direct count is $(t-1)^2$.

Table 1: Retained elementary compositions; every listed input was checked.

Function and parameters	Direct	#out	Ours	Gain	LUT evals.	M
$\lfloor m/d \rfloor$, $m \in [0,63]$, $d \in [1,63]$	4032	64	1192	3.38	3	89
$\lfloor (x_1x_2x_3)^{1/3} \rfloor$, $x_i \in [1,15]$	3375	15	1252	2.70	4	103
$\lfloor a^{1/b} \rfloor$, $a,b \in [2,16]$	225	4	106	2.12	3	15
$\lfloor a^{1/b} \rfloor$, $a,b \in [2,32]$	961	5	234	4.11	3	27
$\lfloor \log_a b \rfloor$, $a,b \in [2,32]$	961	6	347	2.77	3	61
$\lfloor \log(1+x)/\log(1+y) \rfloor$, $t = 32$	961	6	343	2.80	3	60
$\lfloor \log(1+x^2)/\log(1+y^2) \rfloor$, $t = 32$	961	10	435	2.21	3	56

Function and parameters	Direct	#out	Ours	Gain	LUT evals.	M
$\lfloor\sqrt{1+x^2}/\sqrt{1+y^2}\rfloor$, $t=32$	961	22	437	2.20	3	42
$\lfloor\operatorname{asinh}(x)/\operatorname{asinh}(y)\rfloor$, $t=32$	961	5	411	2.34	3	78
$\lfloor\sqrt{\log(1+x)/\log(1+y)}\rfloor$, $t=32$	961	3	339	2.83	3	119
$\lfloor\sqrt[3]{\log(1+x)/\log(1+y)}\rfloor$, $t=32$	961	2	335	2.87	3	175
$\lfloor\sqrt{\operatorname{asinh}(x)/\operatorname{asinh}(y)}\rfloor$, $t=32$	961	3	409	2.35	3	155
$\lfloor\log(1+x)/\log(1+y)\rfloor$, $t=64$	3969	7	913	4.35	3	152
$\lfloor\log(1+x^2)/\log(1+y^2)\rfloor$, $t=64$	3969	12	1187	3.34	3	148
$\lfloor\sqrt{1+x^2}/\sqrt{1+y^2}\rfloor$, $t=64$	3969	45	1311	3.03	3	108
$\lfloor\operatorname{asinh}(x)/\operatorname{asinh}(y)\rfloor$, $t=64$	3969	6	1001	3.97	3	178
$\lfloor\sqrt{\log(1+x)/\log(1+y)}\rfloor$, $t=64$	3969	3	911	4.36	3	303
$\lfloor\sqrt[3]{\log(1+x)/\log(1+y)}\rfloor$, $t=64$	3969	2	913	4.35	3	456
$\lfloor\sqrt{\operatorname{asinh}(x)/\operatorname{asinh}(y)}\rfloor$, $t=64$	3969	3	981	4.05	3	347

The intermediate-route screen also keeps all 19 rows. For example, first forming $x_1x_2x_3$ in the geometric-mean row needs two ciphertext multiplications and leaves a product interval of 3375 values for the final cube-root/floor LUT. Even when those two multiplications are treated as free, the competing LUT has 3375 records, whereas the logarithmic route has 1252. For the bivariate rows, the conservative competitor is one packed-pair LUT; the costs of first evaluating the two transforms are omitted. Thus the displayed Gain is also a lower bound on the record advantage over that competitor.

The last fourteen rows use deeper elementary compositions: their unary feature already contains a logarithm of a logarithm, a square followed by logarithms, or a hyperbolic inverse followed by a logarithm; three rows then add another square root or cube root before flooring. They nevertheless need only two inner unary tables and one outer unary table on the tested domains.

We also tested the suggested trigonometric ratios rather than assuming that they would work. The decompositions were exact, but none reached the 2-fold rule. For

$$\left\lfloor \frac{\sin(\pi x/(2t))}{\sin(\pi y/(2t))} \right\rfloor$$

the ratios were 0.97, 1.07, and 0.94 at (t=16,32,64). The corresponding tangent ratios were 1.13, 1.45, and 0.86, and the cosine ratios were 0.97, 1.07, and 0.94. Adding an outer square root to the sine ratio still gave only 1.18, 1.07, and 0.94. These rows remain in the raw result files. They are a useful warning that greater composition depth and a small output range do not guarantee a short consecutive code interval.

5 Further non-trivial functions

The complete algorithm and every experiment line are available at `https://github.com/3458463857-tech/A-New-Framework-for-Efficient-Multivariate-Functional-Bootstrapping.git`. The repository upload itself is left to the authors; the prepared upload folder contains the exact script, raw outputs, and this English source and PDF.

The comparison here is intentionally harder than Direct versus Ours. For each row we wrote down an obvious route that first constructs a useful intermediate and then applies one final LUT. Ciphertext multiplications in that route are listed in the output file but, conservatively, contribute zero LUT records. We keep a function only when Ours is still strictly smaller than Alt. This is a record-count statement, not a proof of lower latency on every FHE platform.

In the first part of Table 2, $z_{(0)} \le \cdots \le z_{(5)}$ are the sorted inputs for $t = 8, \ell = 6$. The paper's full sorting route uses a verified 12-comparator network, hence $12(2t-1) = 180$ records. The competing route rank-packs all $\binom{t+\ell-1}{\ell} = 1716$ sorted tuples and uses one more LUT, for 1896 records in total. Range and central-four sum need only a seven-comparison min/max network; packing the resulting pair gives the smaller competing count $105 + 8^2 = 169$. The other Alt. counts are stated exactly in the CSV.

For the second part, $c_v = |\{i : x_i = v\}|$, with $t = 5, \ell = 8$. Building all five counts costs $8 \cdot 5^2 = 200$ records in both routes. The competitor then packs the five counts in base 9 and uses one LUT of size $9^5 = 59049$, for 59249

records. Our route instead applies small local tests or count comparisons. Every input tuple was enumerated, and the sorting networks were also checked on all binary inputs by the zero-one principle.

Table 2: Functions retained after the explicit intermediate-route comparison.

Exact function and tested parameters	Direct	#out	Ours	Alt.	Alt./ours	LUT evals.
Maximum $z_{(5)}$	262144	8	75	262144	3495.25	5
Minimum $z_{(0)}$	262144	8	75	262144	3495.25	5
Lower median $z_{(2)}$	262144	8	180	1896	10.53	12
Upper median $z_{(3)}$	262144	8	180	1896	10.53	12
Second smallest $z_{(1)}$	262144	8	180	1896	10.53	12
Second largest $z_{(4)}$	262144	8	180	1896	10.53	12
Range $z_{(5)} - z_{(0)}$	262144	8	105	169	1.61	7
Interquartile range $z_{(3)} - z_{(1)}$ (fixed finite-sample convention)	262144	8	180	1896	10.53	12
Top-two sum $z_{(5)} + z_{(4)}$	262144	15	180	1896	10.53	12
Bottom-two sum $z_{(0)} + z_{(1)}$	262144	15	180	1896	10.53	12
Top-three sum $z_{(3)} + z_{(4)} + z_{(5)}$	262144	22	180	1896	10.53	12
Central-four sum $z_{(1)} + z_{(2)} + z_{(3)} + z_{(4)}$	262144	29	105	169	1.61	7
Median-pair gap $z_{(3)} - z_{(2)}$	262144	8	180	1896	10.53	12
Median absolute deviation: lower median of $\{\lvert x_i - z_{(2)}\rvert : 1 \le i \le 6\}$	262144	4	450	262414	583.14	30
Median-neighborhood count $\sum_{i=1}^{6} \mathbf{1}[\lvert x_i - z_{(2)}\rvert \le 1]$	262144	6	270	262414	971.90	18
Mode, with the smallest symbol chosen on a tie, $t = 5, \ell = 8$	390625	5	268	59249	221.08	44
Distinct count $\sum_{v=0}^{4} \mathbf{1}[c_v > 0]$	390625	5	245	59249	241.83	45
Plurality margin: largest c_v minus second largest c_v	390625	8	353	59249	167.84	49
Modal frequency $\max_v c_v$	390625	7	268	59249	221.08	44
Singleton-symbol count $\sum_v \mathbf{1}[c_v = 1]$	390625	5	245	59249	241.83	45
Repeated-symbol count $\sum_v \mathbf{1}[c_v \ge 2]$	390625	4	245	59249	241.83	45
Exact-two-symbol count $\sum_v \mathbf{1}[c_v = 2]$	390625	5	245	59249	241.83	45
Heavy-symbol count $\sum_v \mathbf{1}[c_v \ge 3]$	390625	3	245	59249	241.83	45
Least positive frequency $\min\{c_v : c_v > 0\}$	390625	5	313	59249	189.29	49
Occupied-frequency range $\max_v c_v - \min\{c_v : c_v > 0\}$	390625	7	381	59249	155.51	53
Number of modal symbols $\sum_v \mathbf{1}[c_v = \max_u c_u]$	390625	4	353	59249	167.84	49

Several formerly reported functions fail this stricter test and are therefore absent from the table. Integer RMS uses 343 records in our coordinate-square route, but only 295 in the optimistic "form the square sum, then bootstrap" route. The corresponding variance counts are 12733 and 442. The Winsorized mean now includes the previously omitted final division LUT: both descriptions use 223 records, so there is no strict advantage. Second moment, collision count, and Gini bucket use 245 records through count LUTs but only 200 if the five ciphertext count multiplications are treated as free. Jaccard, Dice, cosine, and overlap similarly lose by 113 versus 81, 185 versus 153, and 761 versus 729 for the last two. The four dynamic-program examples are also removed because a record-only comparison cannot establish dominance over bit/circuit implementations.

Every value above is generated by `expanded_function_study/compare_intermediate_routes.py` and appears on one line of `expanded_function_study/intermediate_routes/intermediate_routes.csv`. The file also contains the rejected rows, the exact competing route, its LUT evaluations, and the number of ciphertext

multiplications that the conservative comparison ignored. This makes the claim deliberately narrow: the retained functions win this explicit structural record comparison, while ciphertext timings are still needed before claiming an end-to-end speedup.

6 What the experiments suggest

The present data support the following limited conclusions.

1. A much smaller range than domain is the common reason compression is possible. A successful evaluator then exposes a short code interval or a small sufficient state.
2. Low composition depth is a good search guide, especially when logarithms turn products, ratios, variable roots, or variable logarithms into additive features. It is not a guarantee: the tested sine, cosine, tangent, harmonic mean, and two-input geometric-mean cases are concrete counterexamples under the stated metric.
3. The surviving general examples use order statistics or categorical histogram tests. Low-degree moments and normalized set counts are useful intermediates, but in this comparison they favour the competing route.
4. Reachable-state count is not enough. A sparse encoding can still force a large consecutive LUT interval, as the rejected variance experiment shows.
5. The right conclusion is function-by-function and parameter-by-parameter. Structural screening should be followed by a same-platform, correctness-checked TFHE implementation before claiming an end-to-end acceleration.

This is an open empirical catalogue. Later versions may add further elementary compositions, application-specific statistics, or ciphertext measurements, but new rows should use the same rule: publish the definition and code, keep negative results, verify every feasible finite input, and label sampled or model-based evidence plainly.

7 Files and reproduction

From the prepared GitHub folder, run

```
python expanded_function_study/run_experiments.py
python expanded_function_study/compare_intermediate_routes.py
python expanded_function_study/verify_reported_tables.py
```

to regenerate the 76-row base study, the 59 intermediate-route comparisons, and the source-to-table checks. The folder also contains the Chapter 4 compression code, its exact representation records and tests, and the TFHE-rs Table 11 prototype. The implementation for Section 3 is not included in this release and will be open-sourced in a later update.